Legal RAG — Architecture & Design Document

Project: A Retrieval-Augmented Generation (RAG) pipeline that answers natural-language questions about Indian Acts (statutes), grounded in a corpus of 35,892 legal sections drawn from 883 Acts.

Author: Ram **Stack:** Python · PostgreSQL + pgvector · sentence-transformers · Anthropic Claude **Repository:** `legal_rag/`

---

1. Problem Statement

Indian statutory law is voluminous. A single Act (e.g. *Aadhaar Act 2016*) can contain dozens of sections, and a user typically does not know which Section number contains the answer to their question.

Goal: Build a system where a user can ask a question in plain English ("What is the role of UIDAI?") and receive a concise, citation-backed answer drawn only from statutory text — never from the LLM's parametric memory.

This rules out: - Plain keyword search (`grep`-style) — fails on synonyms, paraphrasing. - An LLM alone — it will confidently hallucinate sections that don't exist.

It demands: - **Semantic retrieval** — meaning-based, not keyword-based. - **Grounded generation** — the model is forced to use retrieved text as the only authority. - **Citations** — every answer points back to a chunk so the user can verify.

That combination is exactly what RAG provides.

---

2. What is RAG?

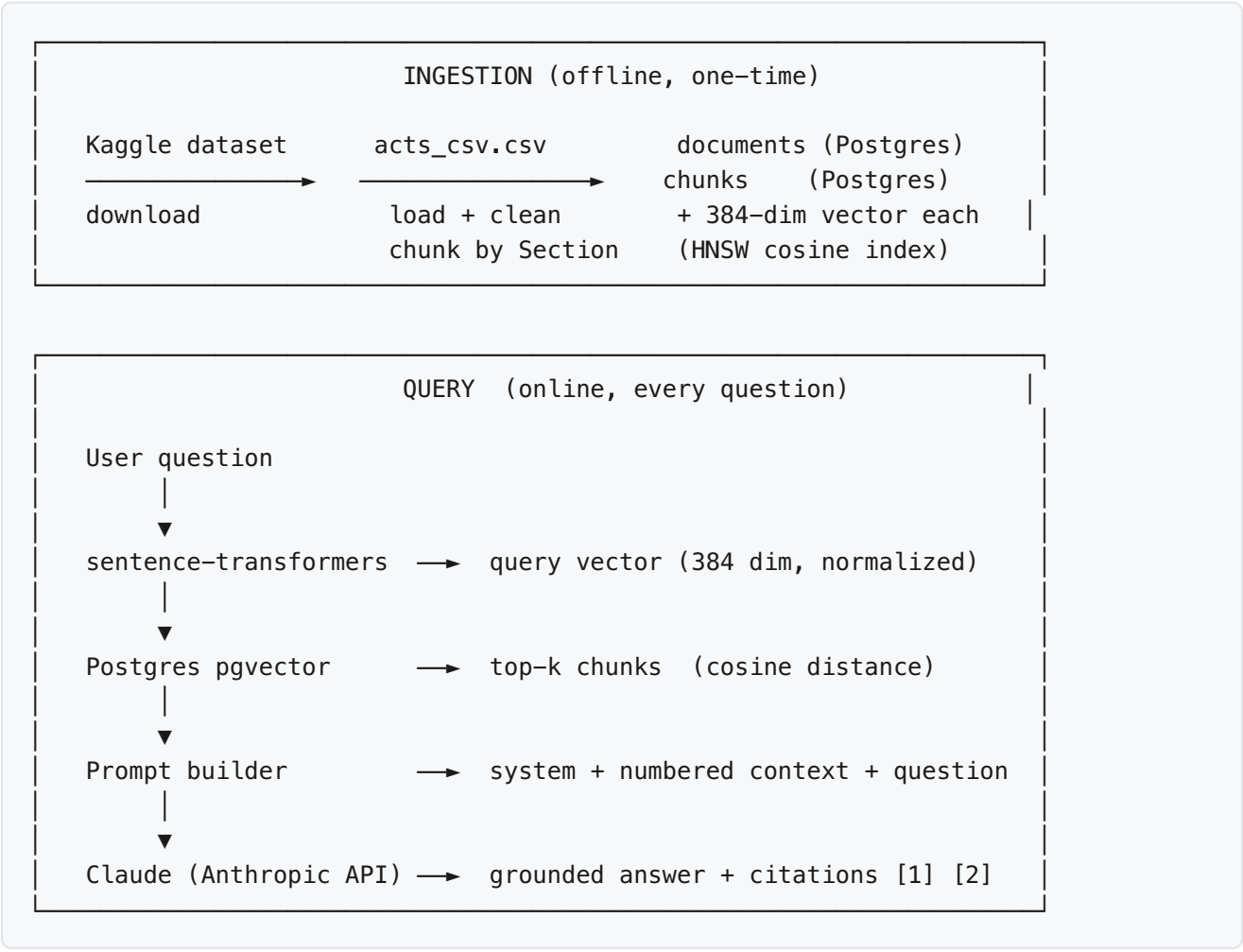
Retrieval-Augmented Generation is a two-stage pattern:

1. **Retrieve.** Convert the user's question into a vector. Find the most semantically similar chunks of text from a pre-indexed corpus.
2. **Generate.** Pass those chunks plus the question to a Large Language Model (LLM) inside a prompt that explicitly says "answer only from the passages below". The LLM's role is *reading comprehension* over the retrieved text, not recall from training.

RAG decouples *knowledge* (which lives in the database, can be updated cheaply) from *reasoning* (which lives in the LLM, swappable). This is why production systems (legal search, customer support, internal docs Q&A) almost universally use RAG instead of fine-tuning.

---

3. High-Level Architecture



The two stages are deliberately separated. Ingestion is expensive (every chunk has to be embedded once) and runs offline. Query is cheap (one embedding + one indexed lookup + one LLM call) and runs in seconds.

---

4. Technology Choices and Justifications

Concern	Choice	Why this, and not the obvious alternative
Vector store	**PostgreSQL + pgvector**	A managed vector DB (Pinecone, Weaviate) would add another service to deploy and bill. Postgres is already a workhorse for transactional data; pgvector adds a `VECTOR(N)` type and an HNSW index in ~50 lines of SQL. One database for documents, metadata, and embeddings simplifies operations.
Embedding model	**`BAAI/bge-small-en-v1.5`** (sentence-transformers, local)	Anthropic does not provide an embeddings API. OpenAI is unavailable in this project. Local sentence-transformers needs no API key, runs offline once downloaded, and `bge-small` is consistently top-tier on the MTEB benchmark for its size class (33M parameters, 384-dim output, ~100MB on disk).
Embedding dim	**384**	Tradeoff: higher dims (1024, 1536) recall slightly better on hard queries but use 4× more storage and slow HNSW search. 384 is the standard "small but capable" sweet spot.
ANN index	**HNSW** with `vector_cosine_ops`	pgvector supports IVFFlat and HNSW. IVFFlat needs a training step on existing data, which complicates the first ingestion. HNSW builds incrementally and gives better recall at the cost of build time / RAM, which is fine for a corpus of this size.
Distance metric	**Cosine distance**	Embeddings from `bge-small` are L2-normalized at inference time (we set `normalize_embeddings=True`), so cosine similarity equals dot product. Cosine ignores magnitude and only measures direction, which is what we want for semantic similarity.
LLM	**Claude Haiku 4.5**	The user has Anthropic API access. Haiku is the smallest / cheapest / fastest tier of Claude — sufficient for reading-comprehension tasks once the right context is provided.
Chunking strategy	**Section-level (already in the dataset)**	The Kaggle dataset is pre-chunked per statutory Section. A Section is the natural unit of legal citation, so splitting on character count would *destroy* meaningful boundaries. We trust the dataset's existing structure.
Why not LangChain / LlamaIndex?	Not used	These libraries hide the four moving parts (chunk, embed, retrieve, generate) behind layers of abstraction. For a learning project and viva, building each piece directly with `psycopg`, `sentence-transformers`, and `anthropic` makes the data flow visible. The whole pipeline is < 250 lines.

---

5. Database Design

5.1 Tables

```
CREATE EXTENSION IF NOT EXISTS vector;        -- registers the VECTOR(N) type

CREATE TABLE documents (
    id           BIGSERIAL PRIMARY KEY,
    source       TEXT NOT NULL,               -- e.g. "kaggle:aayaniqbal2005/..."
    title        TEXT,                        -- "Aadhaar Act 2016"
    raw_text     TEXT,                        -- full text (nullable)
    metadata     JSONB NOT NULL DEFAULT '{}',
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE chunks (
    id           BIGSERIAL PRIMARY KEY,
    document_id  BIGINT NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    chunk_index  INT NOT NULL,
    chunk_text   TEXT NOT NULL,
    embedding    VECTOR(384),                 -- 384 = bge-small output dim
    token_count  INT,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (document_id, chunk_index)
);

CREATE INDEX chunks_embedding_hnsw
    ON chunks USING hnsw (embedding vector_cosine_ops);
```

5.2 Why two tables (documents, chunks) instead of one?

Normalization. Each Act has a single name ("Aadhaar Act 2016") shared across all its sections. Storing that name 50 times — once per section — would denormalize the data and bloat the table. The foreign key `chunks.document_id → documents.id` is the standard relational fix.

Re-chunking. If we ever change the chunking strategy (e.g. switch from "one section per chunk" to "two sections sliding window"), we keep `documents` intact and only rebuild `chunks`. This separation matters more on dynamic corpora.

Citation. Joining `chunks` back to `documents` lets us cite the answer as "*Aadhaar Act 2016, chunk #14*" instead of the opaque chunk id.

5.3 The HNSW index — what it actually does

A vector similarity search of the form

```
SELECT * FROM chunks ORDER BY embedding <=> $1 LIMIT 5;
```

is conceptually an O(N) scan: compute distance to every row, sort, take top 5. With 35,892 chunks of 384 dims that is 13.7 million floats, multiplied by a few additions and one square root per chunk. Acceptable today, slow at 10× scale.

HNSW (Hierarchical Navigable Small World) is an Approximate Nearest Neighbour (ANN) index. It builds a multi-layer graph where each node connects to a few neighbours; search starts at the top sparse layer, greedily moves towards the query, then descends one layer at a time. Search complexity is roughly O(log N).

Approximate means recall is ~95–99 % rather than 100 %. For semantic search this is invisible to the user — the difference between the 5th and 6th nearest neighbour is rarely meaningful.

The operator class `vector_cosine_ops` tells pgvector that distance is `1 − cos(θ)` between the query and indexed vectors. Since we normalize embeddings, this also equals `(2 − 2·dot_product) / 2`, just at a constant scale.

---

6. The Embedding Pipeline

6.1 What is an embedding?

A function `f : text -> R^384` that maps semantically similar texts to nearby points in a 384-dimensional vector space. Trained on hundreds of millions of (sentence, paraphrase) pairs.

For our model, `f("UIDAI is the Unique Identification Authority of India")` and `f("the Authority that issues Aadhaar numbers")` end up only ~0.15 cosine distance apart, while `f("the moon is made of cheese")` is ~0.85 away from both.

6.2 The chunk-text we actually embed

Before embedding, each chunk row is built as:

```
"<Act name>, Section <N>\n<original text>"
```

Concretely: `"Aadhaar Act 2016, Section 14\nThere shall be the Authority by ..."`

Why prepend the Act name and Section number? Embeddings are sensitive to all the text they see. When a user asks "*what does the Aadhaar Act say about the Authority?*", the chunks whose embedded text already contains the literal phrase "Aadhaar Act" score systematically higher. This is a free retrieval boost and costs nothing at inference time.

6.3 Batched embedding

`sentence-transformers` is dramatically faster on batches than one item at a time, because the GPU/CPU vectorizes the matrix multiplications across the batch dimension. We embed in batches of 64, which is a sweet spot for `bge-small` on Apple Silicon CPU.

```
vecs = model.encode(
    texts,
    batch_size=64,
    normalize_embeddings=True,    # unit vectors -> cosine == dot product
    convert_to_numpy=True,
)
```

In our run on M-series Mac CPU, the first batch is slow (~25 s) due to model loading and warmup; subsequent batches process at roughly 60 chunks/second.

---

7. The Retrieval Stage

7.1 The SQL

```
SELECT c.document_id,
       d.title,
       c.chunk_index,
       c.chunk_text,
       c.embedding <=> $1 AS distance     -- pgvector cosine distance
FROM chunks c
JOIN documents d ON d.id = c.document_id
ORDER BY c.embedding <=> $1               -- HNSW kicks in here
LIMIT $2;
```

The `<=>` operator is overloaded by pgvector:

Operator	Meaning
`<->`	Euclidean (L2) distance
`<=>`	Cosine distance
`<#>`	Negative inner product

We use `<=>` and our HNSW index was built with `vector_cosine_ops`, so the planner uses the index and the search is sub-linear.

7.2 Why top-k and not a similarity threshold?

A fixed threshold (e.g. "include all chunks with distance < 0.4") is fragile because absolute distance values shift with the embedding model and even with the query. Top-k with k=5–10 is a robust default; if no chunks are relevant, the LLM is instructed to say so.

7.3 What `Hit` looks like to the rest of the code

```
@dataclass
class Hit:
    document_id: int
    title: str
    chunk_index: int
    chunk_text: str
    distance: float
```

The `retrieve.search()` function returns `List[Hit]`, which is then formatted into a numbered context block for the LLM.

---

8. The Generation Stage

8.1 The prompt

Three pieces, concatenated:

1. **System prompt.** Tells Claude its role and its hard constraints: ``` You are a legal assistant answering questions about Indian Acts.

Rules: - Use ONLY the numbered context passages provided. - If the context does not contain the answer, say so explicitly. Do not guess. - Cite passages by their number, e.g. "[1]", "[2]". - Keep answers concise and quote the relevant statutory text when helpful. ```

1. **Context block.** The retrieved chunks, numbered: ``` [1] Aadhaar Act 2016, chunk #14

[2] Aadhaar Act 2016, chunk #20 ... ```

1. **User question.** Verbatim.

8.2 Why the system prompt matters more than people think

Without the "use ONLY the context" instruction, Claude would happily blend its training knowledge with the retrieved chunks. The result might be fluent and correct *most* of the time, but you cannot tell when it has drifted off-source. For a legal application that is unsafe.

The phrase **"If the context does not contain the answer, say so explicitly"** is what gives the system honest behaviour on out-of-corpus questions. It is the difference between a system you can defend and one you cannot.

8.3 Choice of model

Haiku 4.5 is the smallest tier of Claude. Two reasons: - **Cost.** Haiku is roughly 10× cheaper than Sonnet for input tokens. - **Latency.** Haiku produces tokens faster.

For RAG, the bottleneck in answer quality is *retrieval*, not generation. Once the right 5 chunks are in the prompt, even a small model writes a decent answer. Spending the budget on a bigger LLM is rarely the highest-leverage move.

---

9. End-to-End Walk-through

A worked example with the question "*What is the role of UIDAI under the Aadhaar Act?*".

Step 1. `scripts/ask.py` parses the CLI argument, calls `rag.ask(question, k=5)`.

Step 2. `rag.ask` calls `retrieve.search(query, k=5)`.

Step 3. `retrieve.search` calls `embed.embed([query])`. Behind the scenes: - `_get_model()` lazy-loads bge-small from `~/.cache/huggingface/`. - The query is tokenized, passed through the transformer, and the output is mean-pooled and L2-normalized. - We get a `(1, 384) float32` numpy array.

Step 4. `retrieve.search` opens a pgvector connection (`db.get_conn()`), runs the SQL above, gets 5 rows.

Step 5. Each row is wrapped in a `Hit` dataclass. Distances were 0.2230, 0.2484, 0.2543, 0.2668, 0.2714 in the test run — all from the *Aadhaar Act 2016*, which is correct.

Step 6. `rag.ask` formats the context block, builds the prompt, calls `anthropic.Anthropic().messages.create(...)`.

Step 7. Claude's response — text only, citations like `[1]`, `[2]` — is returned alongside the original `Hit` objects so the caller can render the source list.

Step 8. `scripts/ask.py` prints the answer and the sources.

Total wall time on a warm cache: ~1–3 s (most of it the LLM call). Total cost: roughly 1500 input tokens + 500 output tokens of Haiku, less than a tenth of a US cent.

---

10. Code Organization

```
legal_rag/
├── .env                        # ANTHROPIC_API_KEY, KAGGLE_API_TOKEN, PG vars  (gitignored)
├── .env.example                # template (committed)
├── requirements.txt            # python deps (psycopg, pgvector, anthropic, sentence-transfor
├── sql/
│   └── 01_schema.sql           # CREATE EXTENSION vector + documents + chunks + HNSW index
├── src/
│   ├── config.py               # loads .env, exposes typed settings (one source of truth)
│   ├── db.py                   # get_conn() context manager + pgvector type registration
│   ├── embed.py                # lazy-loaded sentence-transformers wrapper
│   ├── retrieve.py             # search(query, k) -> List[Hit]
│   └── rag.py                  # ask(question) -> RagAnswer (calls retrieve, then Claude)
├── scripts/
│   ├── ingest.py               # CSV -> documents + chunks (with embeddings)
│   └── ask.py                  # CLI: python -m scripts.ask "your question"
└── data/
    └── raw/
        └── acts_csv.csv        # 35,892 rows, ~40MB, downloaded from Kaggle
```

Module responsibilities (single-responsibility per file)

File	One sentence
`config.py`	Read `.env`, fail loudly if a required key is missing, expose constants.
`db.py`	Give callers a `with get_conn() as conn:` context manager that already knows about `VECTOR`.
`embed.py`	Turn `List[str]` into `np.ndarray[float32]` of shape `(N, 384)`.
`retrieve.py`	Given a question and k, return the top-k most similar `Hit`s.
`rag.py`	Stitch retrieve + Claude together with the right prompt; return `RagAnswer`.
`ingest.py`	Read CSV, insert acts into `documents`, embed and insert sections into `chunks`.
`ask.py`	Thin CLI wrapper around `rag.ask`.

This layering means each file has at most one external dependency (`config.py` knows about no other module; `db.py` only imports `config`; `embed.py` only imports `config`; etc.), which makes the data flow easy to trace.

---

11. Limitations and Honest Tradeoffs

A viva will ask "what's wrong with your system?" — be ready.

1. The corpus is small (currently 30 acts of 883). The current ingestion is a sample for demonstration. A full ingestion is a one-line change (`python -m scripts.ingest --limit 0`) and runs in roughly 30 minutes on CPU. The architecture itself does not change.

2. We do not detect when retrieval has failed. If the user asks about an act that is not in the corpus, the top-k search still returns 5 chunks — they just have higher cosine distance. We rely on Claude to notice and say "the context does not contain this." A more rigorous system would set a distance threshold or train a separate "is this query answerable from the corpus" classifier.

3. No re-ranking. Top-k from cosine similarity is a *first-stage* retrieval. State-of-the-art systems re-rank the top-20 with a cross-encoder (which sees query+chunk together) before passing 5 to the LLM. We skipped this for simplicity. It is the single highest-leverage upgrade.

4. No hybrid search. "Section 5 of the Aadhaar Act" is a partly-keyword query. Pure semantic search may rank a section about *content* of section 5 above the literal section 5. Hybrid retrieval (BM25 + vector, fused with reciprocal rank fusion) handles this.

5. Single-turn only. The user cannot follow up ("what about subsection 2?") because we do not pass conversation history into the retriever. Multi-turn RAG requires query reformulation: rewrite the user's follow-up using the chat history, then retrieve.

6. Embedding model is general-purpose, not legal. `bge-small-en-v1.5` was trained on web/Wikipedia text. A model fine-tuned on legal text (Voyage's `voyage-law-2`, or `legal-bert`) would likely score 5–10 % higher on retrieval recall. The wiring is identical; only the model name and dim change.

7. Chunks can be huge. The dataset has chunks up to 141,843 characters. We do not split these further. For long sections, a finer-grained sub-chunking would help (the 5 retrieved chunks all together fit Claude's context window today, but on a different LLM that might break).

8. No evaluation harness. We have no automated way to measure retrieval@k or answer faithfulness. A real project would maintain a held-out set of (question, gold answer, gold sections) triples and report numbers. The Kaggle `IndicLegalQA` dataset is a candidate.

---

12. Possible Extensions (the "what would you build next" question)

Priority	Extension	Effort	Expected gain
1	Cross-encoder re-ranker on top-20	small (one new module)	Big — usually 10-20% recall@5 jump
2	Hybrid search (BM25 + vector)	medium (Postgres `tsvector` + RRF in SQL)	Strong on keyword-heavy queries
3	Query rewriting for multi-turn	small (one prompt + history slice)	Enables real conversational use
4	Streaming answer with sources first	small (use `client.messages.stream`)	UX polish
5	Switch to `voyage-law-2` embeddings	small (change model + EMBED_DIM, re-ingest)	Better legal retrieval
6	Evaluation harness with `IndicLegalQA`	medium (script + 50–100 labeled Qs)	Lets you measure changes
7	Web UI (FastAPI + minimal HTML)	medium	Demo-able

The order above reflects ROI per hour of work, not novelty. Re-ranking and hybrid search are the two upgrades that almost every production RAG eventually adds.

---

13. Likely Viva Questions and Sample Answers

These map directly to sections above. Skim them before the viva.

Q1. Walk me through your architecture. Two phases. *Ingestion* is offline: a Kaggle CSV with one row per Section is loaded, each row becomes a chunk, each chunk is embedded with sentence-transformers `bge-small` and inserted into Postgres. *Query* is online: the user's question is embedded with the same model, pgvector returns the top-k most similar chunks via an HNSW index using cosine distance, those chunks are packed into a prompt that instructs Claude to answer only from them, and the final answer is returned with citations.

Q2. Why pgvector instead of Pinecone or FAISS? pgvector co-locates vectors with the rest of the data — documents, metadata, foreign keys — in the same Postgres database. That removes a service from the deployment and makes JOINs trivial. FAISS is in-memory and would need to be rebuilt on every process start. Pinecone is excellent but adds a billed external dependency. For corpus sizes up to a few million chunks, pgvector is sufficient and operationally simpler.

Q3. What is HNSW and why use it? HNSW = Hierarchical Navigable Small World. It is an approximate nearest-neighbour index built as a multi-layer graph. Search starts at the sparsest top layer, greedily walks towards the query, then descends. It gives logarithmic-time search at ~95-99% recall. The alternative in pgvector is IVFFlat, which requires a training step on existing data and is harder to operate during the first ingestion.

Q4. What does cosine distance measure? The angle between two vectors, ignoring magnitude. Specifically `1 − (A·B)/(|A||B|)`. With L2-normalized embeddings, `|A| = |B| = 1`, so cosine distance reduces to `1 − A·B`. Smaller is more similar. We use it because semantic similarity should be invariant to text length, which determines vector magnitude.

Q5. How does your system avoid hallucinations? Three mechanisms working together. (a) *Retrieval-first*: the LLM never answers from raw weights — it always sees explicit context. (b) *System prompt*: explicitly tells Claude to use only the provided passages and to say so when the answer is not present. (c) *Citations*: the answer must reference passage numbers `[1]`, `[2]`, which lets the user verify. The boundary test in the demo (asking about the Income Tax Act, which is not in the corpus) confirms the model declines to answer rather than hallucinating.

Q6. Why do you embed `"Act name, Section N\nText"` instead of just `Text`? Embeddings are sensitive to all the text they see. Including the Act name and section number gives the embedded vector explicit "metadata" that the user's natural-language query can match against. When the user asks "*what does the Aadhaar Act say about authentication?*", the chunks whose embedded text starts with "*Aadhaar Act 2016*" score systematically higher.

Q7. What is the role of `normalize_embeddings=True`? It makes every output vector unit-length. With unit vectors, cosine distance equals `1 − dot_product`, so the underlying math becomes a simple dot product. This is faster, and pgvector's HNSW with `vector_cosine_ops` is optimized for this case.

Q8. How would you handle the 35,892-chunk full corpus? The code already handles it — `python −m scripts.ingest −−limit 0`. CPU embedding takes about 30 minutes. The HNSW index continues to give logarithmic-time search; the only operational change is that loading the model into memory takes longer relative to small batches.

Q9. What happens if pgvector is asked for a chunk count larger than what exists? It returns however many it has, with no error. The `LIMIT 5` is a cap, not a requirement. The retriever is robust to this.

Q10. Why Claude Haiku and not Sonnet or Opus? Cost-latency tradeoff. RAG quality is dominated by retrieval, not generation. Once the right context is in front of any modern LLM, the reading-comprehension task is easy. Haiku is roughly 10× cheaper and faster than Sonnet for the same answer quality on grounded Q&A. If the retrieval were poor, a bigger model would not save us.

Q11. What is your most important limitation? No re-ranking. Top-k cosine is a first-stage filter; a cross-encoder re-ranker (which sees query + chunk together) would deliver the single largest accuracy jump. Adding it is one module, ~50 lines.

Q12. How would you scale this to 10 million chunks? Three orthogonal steps. (a) Switch the index to IVFFlat with a sensible `lists` parameter (sqrt of N). (b) Move embedding to a GPU or batched API. (c) Partition the `chunks` table by `document_id` range. The application code is unchanged.

Q13. Why two tables (documents + chunks) instead of one wide table? Normalization. Each document has a name shared by all its chunks; storing it once avoids 50× duplication. Foreign keys make ON DELETE CASCADE behaviour clean. And re-chunking only touches the `chunks` table.

Q14. How do you keep the pipeline reproducible? Pinned versions in `requirements.txt`. A `.env.example` template. The schema as `sql/01_schema.sql`. The ingestion script wipes the tables

and re-runs from the CSV. Anyone with the repo, a Kaggle key, and an Anthropic key can stand up an identical pipeline in under 5 minutes.

Q15. Could you explain `VECTOR(384) USING hnsw (embedding vector_cosine_ops)`? `VECTOR(384)` is a pgvector column type — a fixed-length array of 384 floats. `USING hnsw` says use an HNSW graph index instead of B-tree (which makes no sense for vectors). `vector_cosine_ops` is the operator class that tells pgvector to interpret distance as cosine, which is what the HNSW graph will optimize for at search time.

---

14. References

- pgvector — https://github.com/pgvector/pgvector
- BGE embedding model — https://huggingface.co/BAAI/bge-small-en-v1.5
- HNSW paper — Malkov & Yashunin, *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs* (2018)
- MTEB benchmark (embedding leaderboard) — https://huggingface.co/spaces/mteb/leaderboard
- Anthropic Claude API — https://docs.anthropic.com
- Kaggle dataset — https://www.kaggle.com/datasets/aayaniqbal2005/indian-supreme-court-judgments-section-wise

---

End of document.